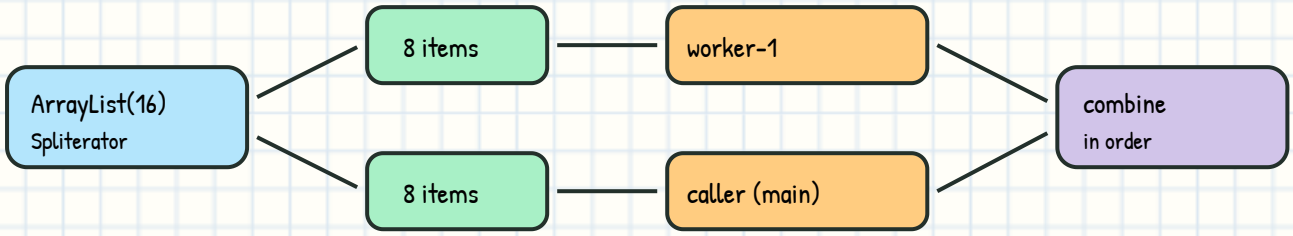
Parallel streams · my notes L14

more workers is not always faster

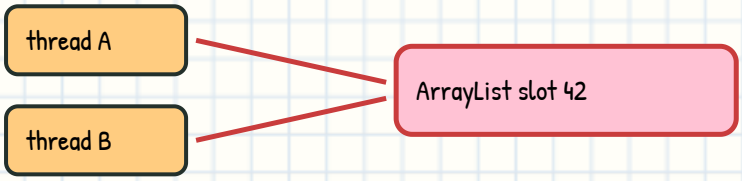
Split, run, combine 🍕



LinkedList(16) → 16 + 0: no real split

commonPool = cores − 1

Shared list = lost items 💥



95301 · 94292 · 100000

same code, 3 runs!

fix: .toList() → always 100000

reduce rules ⚖

associative: (a−b)−c ≠ a−(b−c) ×

1..10 minus: −55 seq, parallel differs

identity neutral: reduce(10, +) → 65 seq, more in parallel

fix: 10 + sum() → 65 ✓

Order 📋

`toList()` keeps order ✓

`forEach` any order

`forEachOrdered` order, slower

`findFirst` → P005 always

`findAny` → P005/P008/P010

One pool per JVM 🏊

all parallel streams share `commonPool`

+ CompletableFuture async (no executor)

a blocking call parks a shared worker

Last call wins 🚦

`.parallel().filter(..).sequential()`

→ isParallel() = false

one flag for the WHOLE pipeline

CPU vs waiting 🧠⏳

parallel stream [8]

CPU work, data in memory

few platform threads (commonPool)

large N × Q, measured with JMH

blocking inside → starves the pool

virtual threads [21]

many blocking tasks (HTTP, JDBC)

one cheap thread per task

12 enrichments: in order, all virtual

waiting is cheap, CPU is not added

Should it be parallel? ✅

12 payments → NOT (N × Q too small)
5M prices × heavy maths → candidate: measure
10k REST calls → NOT: virtual threads
shared HashMap → NOT: fix first

Measure right ⏱

JMH: warm-up, forks, return results
realistic data, target hardware
one nanoTime ×

Worked example: fee report

loop · parallel `collect(groupingBy)` with a pure fee() · `unordered + groupingByConcurrent + teeing`, sorted at the edge → TRANSFER 8 debits 17.97 … TOTAL FEES 42.95, every run ✓

Exercise: fix 3 broken pipelines 🔧

reduce(float, add) → float.add(reduce(ZERO, add)): 4529.84 · findAny → findFirst: P005 · shared ArrayList → toList(): [P001, P002, P004, …] · tests repeat 50×

★ Remember

sequential by default · no shared state · associative + neutral identity · findFirst for "first" · never block the common pool · JMH, not guesses · next → L15 FP principles (Module 3)